Rapport de projet : Système de QCM client/serveur exploitant une file MQTT

Groupe : S5-1A

Matière : Info0502 Programmation répartie

Professeur référent : Olivier Flauzac

Etudiant : Joseph Geeraerts

1. Architecture générale du projet

- Serveur MQTT : Mosquitto.
- Serveur gestion des QCM, traite les requêtes reçus.
- Client, envoie requêtes vers le serveur.

L'architecture du projet repose sur une structure Gradle multi-modules. Ce projet principal est composé de deux sous-projets distincts : l'un dédié au client, et l'autre au serveur.

Création du projet gradle :

```
mkdir info0502-projet-qcm
cd info0502-projet-qcm
gradle init --type basic --dsl groovy
mkdir QCM_Manager
mkdir Client1
```

Ajouter à settings.gradle :

```
include 'QCM_Manager'
include 'Client1'
```

Initialiser sous-projets avec gradle :

```
cd QCM_Manager
gradle init --type java-application --dsl groovy
cd Client1
gradle init --type java-application --dsl groovy
```

Le développement a été centralisé sur une seule machine afin de simplifier le versioning via Git. Le code est écrit et poussé (*push*) depuis la machine serveur, puis récupéré (*pull*) sur la machine cliente pour tester le fonctionnement du code. Dans une optique de déploiement final, il faudrait supprimer le projet Client sur le serveur, et le projet Serveur sur la machine cliente.

Service installé sur la machine Serveur MQTT :

1. Mosquitto : sudo apt-get install mosquitto

Les services installées sur les machines, cliente et serveur :

1. **Installation de wget :** sudo apt-get install wget
2. **Installation du JDK 25 :** Téléchargement de l'archive via wget.

- Extraction : tar -xvf [nom_archive] et déplacement vers /opt.
- Configuration des variables d'environnement dans .bashrc :

```
export JAVA_HOME=/opt/jdk-25.0.1
export PATH=$JAVA_HOME/bin:$PATH
```

3. **Installation de Gradle version 9.2.1 :**

   - Installation de unzip : sudo apt install unzip
   - Téléchargement et extraction vers /opt.
   - Configuration du Path dans .bashrc :

```
export GRADLE_HOME=/opt/gradle-9.2.1
export PATH=$GRADLE_HOME/bin:$PATH
```

4. **Accès distant :** Installation d'OpenSSH (sudo apt-get install openssh-server) sur les machines. Pour l'accès à distance via VS code, add configuration (ssh @ip_machine).

Implémentation des différents acteurs :

1) Le serveur

Le serveur va servir à répondre aux différentes requêtes du client. l'inscription, l'authentification, la demande d'exécution d'un QCM...

- Communication avec les clients :

  Le serveur utilise la bibliothèque Mosquitto (via Paho) pour se connecter au broker. Il s'abonne uniquement au topic server/qcm. Il écoute en permanence les requêtes publiées par les clients sur ce canal. Lorsqu'il doit répondre à un client spécifique, il récupère dans la requête l'id unique du client puis publie le message de retour sur le topic spécifique (client_id/res).

  Pour garantir l'unicité des échanges, chaque client génère un identifiant unique (UUID) au démarrage. Le serveur utilise cet identifiant pour publier ses réponses sur un topic dédié (client_id/res). Ce mécanisme assure que chaque message est routé vers la bonne instance du client, évitant ainsi les conflits si lors d'une inscription, le nom renseigné par l'utilisateur est déjà pris.

- Traitement des messages (APIMessage) :

  Le traitement des requêtes est géré par la classe APIMessage. Lorsqu'un message JSON arrive, il est désérialisé grâce à la bibliothèque Google Gson. Le serveur analyse le champ action de la requête (ex: REGISTER, LOGIN, DEMANDE_EXEC_QCM) et redirige le traitement vers la méthode appropriée via un switch case.

- Gestion des Données :

  Le serveur doit stocké 2 informations importantes, les QCM, et les données clients.

  - UserService : Gère les utilisateurs et leurs sessions. Les données des utilisateurs (login, mot de passe haché, historique QCM) sont stockées de manière persistante dans un fichier JSON. Il gère également une Map des sessions actives pour identifier les utilisateurs connectés.

  - Gestionnaire de QCM : Charge et stocke les QCM disponibles. Lors de l'exécution d'un QCM, le serveur gère la progression en envoyant les questions une par une et en corrigeant les réponses au fil de l'eau pour mettre à jour le score de la session.

2) Le client

Le client va permettre de contacter le serveur, il faut donc une interface qui permet à l'utilisateur d'envoyer des requêtes au serveur ex (Register, Login, Profil, Liste de QCM ...) :

- Identification et Connexion :

  Au démarrage, le client génère un identifiant unique (UUID). Il s'abonne immédiatement à son topic de réponse personnel client_id/res. Toutes les requêtes (inscription, login ...) sont envoyées au topic global server/qcm.

- Envoie de requêtes :

  Pour simplifier l'envoies des requêtes au serveur on crée une classe requête qui comporteras la structure de toutes nos requêtes :

  - sessionId (donner lors du login)
  - action (pour que le serveur puisse identifier l'action à réaliser)
  - payload (le payload sera remplis en fonction de l'action)

  Afin de structurer le contenu du payload, nous avons implémenté des classes spécifiques (telles que RegisterPayload ou LoginPayload) regroupées dans un module partagé (common).

  Cette architecture permet au client et au serveur d'utiliser les mêmes définitions de classes Java. L'avantage majeur est la cohérence des données. Le serveur peut manipuler directement des objets typés via les *getters*, ce qui élimine les risques d'erreurs liés au parsing manuel.

  L'envoi des requêtes est gérer par un menu textuel interactif. L'utilisateur déclenche les actions via des touches spécifiques (par exemple 'r' pour l'inscription, 'l' pour la connexion), ce qui génère et expédie la requête appropriée au serveur.

- Gestion de la Synchronisation :

  Nous devons réaliser un modèle Client Serveur en utiilisant une file MQTT, il faut donc pouvoir organisé la réception des réponses du serveur. Pour résoudre ce problème, nous avons implémenté le modèle Producteur-Consommateur avec une BlockingQueue.

  - **Le Producteur (Thread MQTT) :** Lorsqu'une réponse arrive du serveur sur le topic MQTT, il dépose le message JSON dans la file d'attente interne (responseQueue.offer()) et se libère immédiatement.

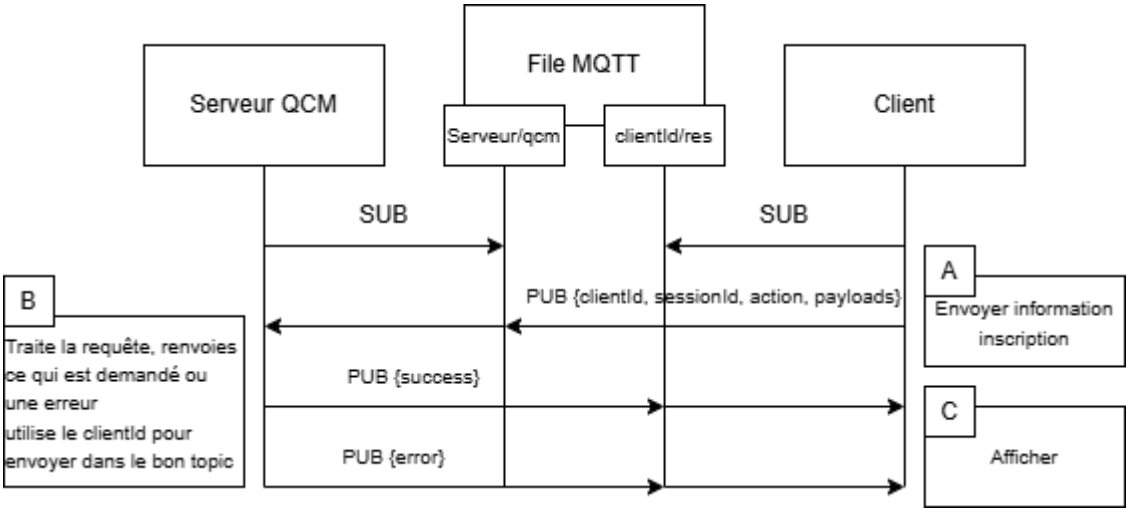
  - **Le Tampon (BlockingQueue) :** Cette file agit comme une zone tampon sécurisée (thread-safe) entre la réception réseau asynchrone et le traitement logique.

  - **Le Consommateur (Thread Principal / Main) :** Après avoir envoyé une requête, le thread principal se met volontairement en pause en appelant la méthode bloquante responseQueue.take(). Il reste bloqué jusqu'à ce que le Thread MQTT dépose une réponse dans la file. Cela garantit que l'affichage du menu ou de la question suivante ne se fait qu'une fois les données effectivement reçues.

2. Description de l'analyse du projet :

A. Chronogramme de modélisation :

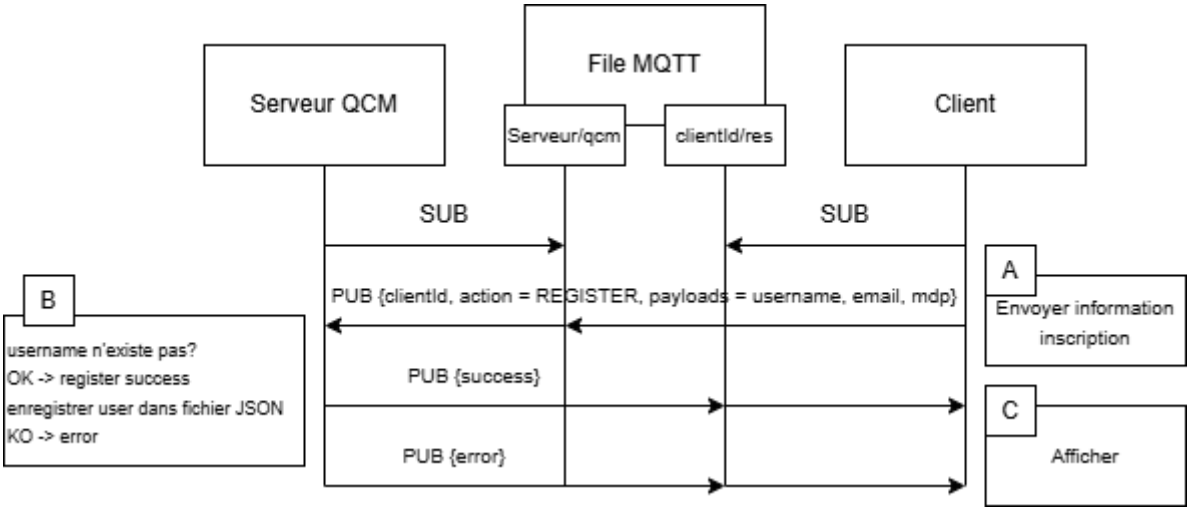
Représentation globale des échanges :



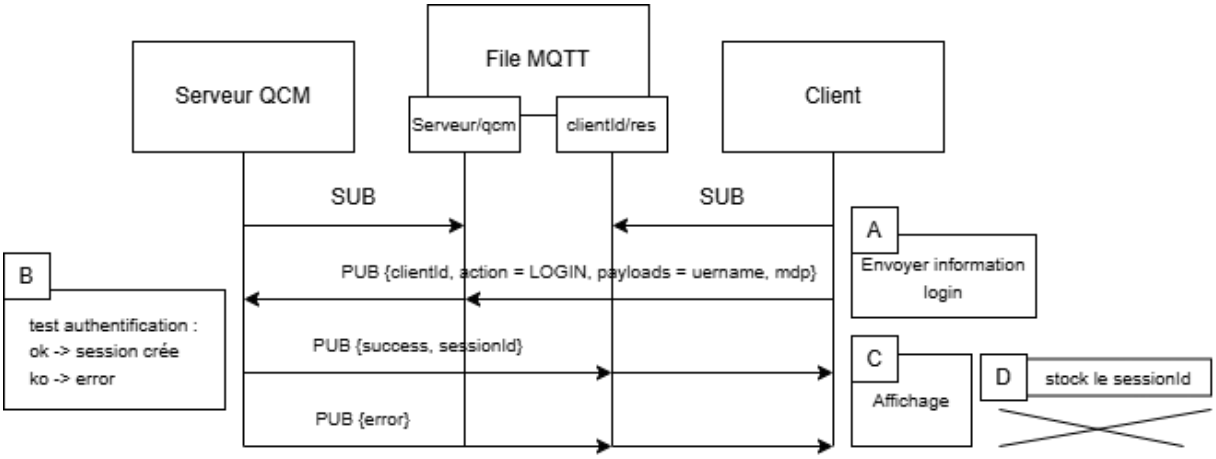
Le Serveur QCM s'abonne à son topic, dans lequel le client va publier ses requêtes. Et le client s'abonne à son topic clientId, dans lequel le serveur publieras la réponse à la requête du client.

Le serveur reprend l'id unique du client contenu dans toutes les requêtes pour créer dynamiquement le bon topic de réponse.

Modélisation Register (On vérifieras que le username n'est pas déjà enregistré) :

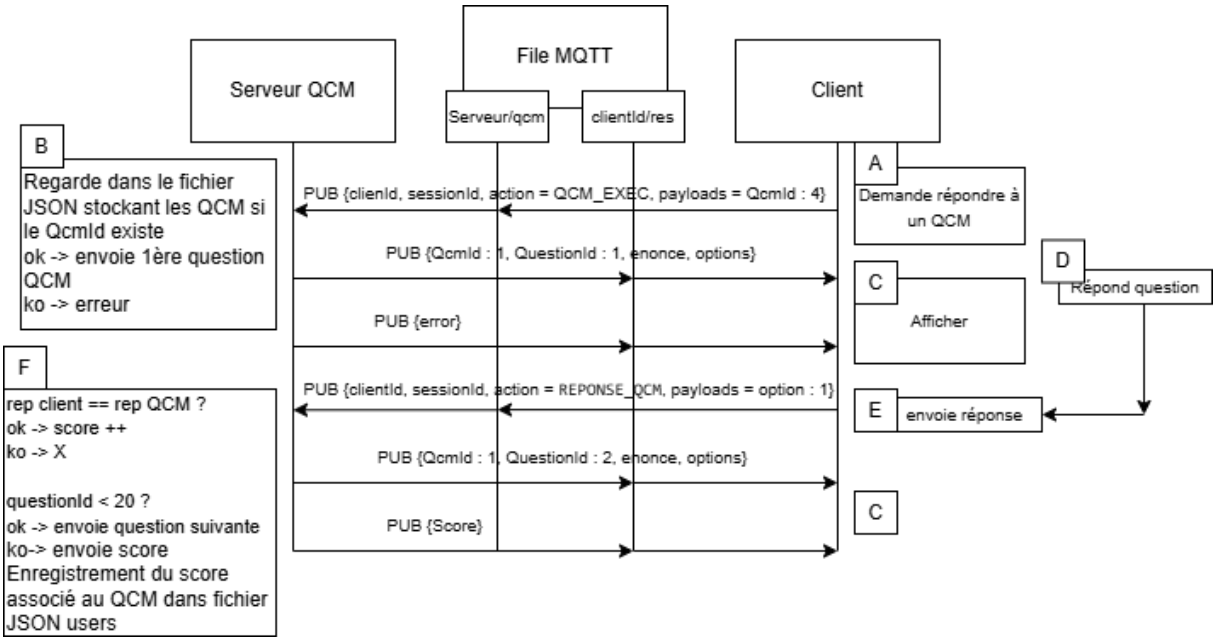


Modélisations Login :



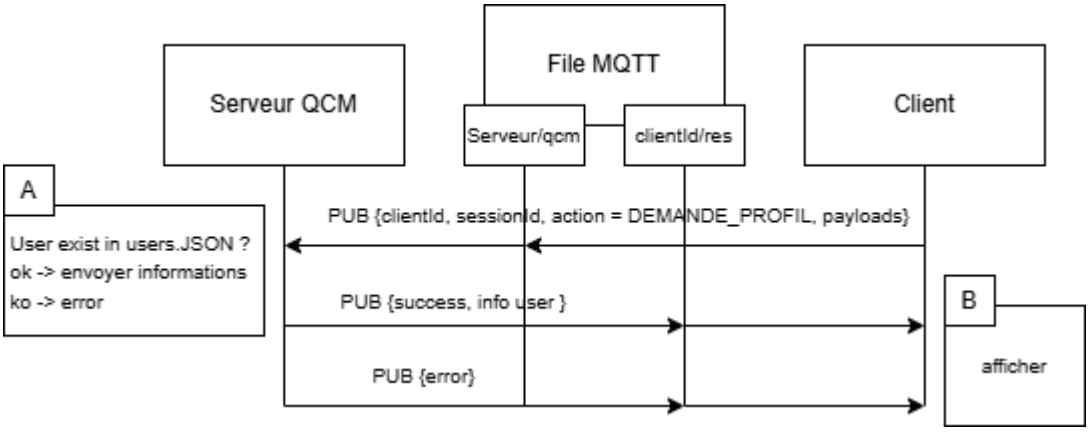
On se connecte en utilisant son nom d'utilisateur et son mot de passe, puis un idSession nous est attribué, et à chaque échange l'idSession sera vérifié.

Modélisation exécution du QCM :

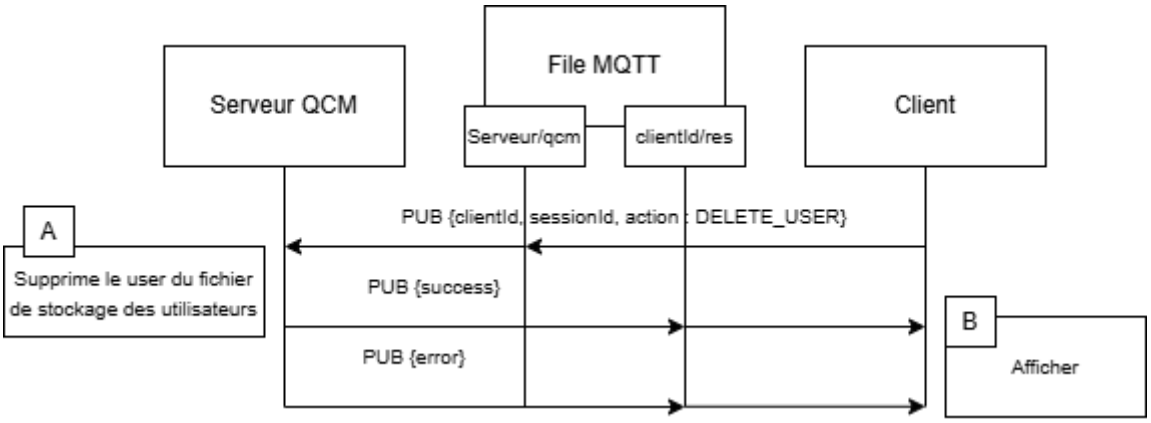


On commence par demandé un QCM avec un id précis, puis le serveur nous envoie la question, l'utilisateur réponds, et ainsi de suite jusqu'à la 20 -ème question, où le serveur renverras le score.

Modélisation voir Profil :

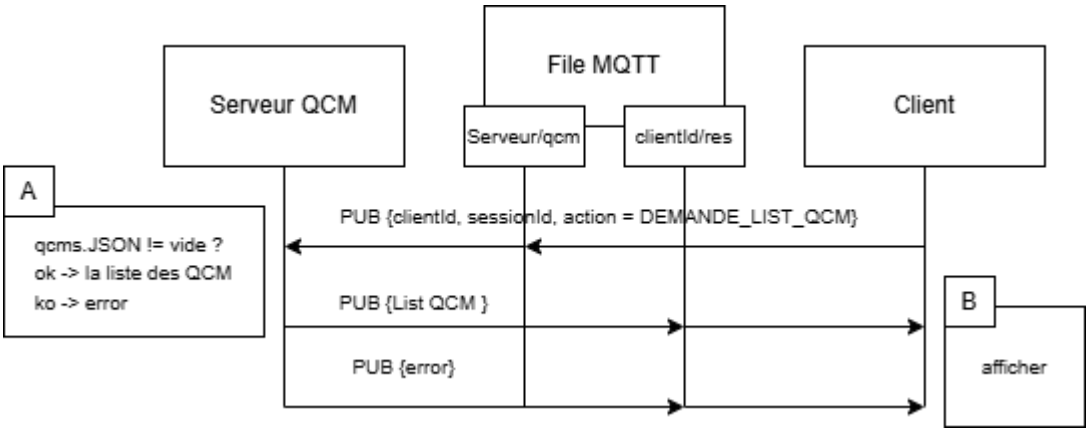


Modélisation suppression compte :



Lorsque le compte est supprimé, alors le client est déconnecté du serveur, il doit donc se reconnecter avec un autre compte pour réaliser de nouveaux QCM.

Modélisation voir QCM disponible :



B. Représentation des données en JSON :

Requête d'inscription :

```
{
  "clientId": "uuid-aleatoire",
  "sessionId": "",
  "payload": {
    "username" : "user1",
    "email": "fake@email.fr",
    "password": "mdp"
  }
 }
```

Réponse :

```
{
  "status": "OK",
  "action": "REGISTER_SUCCESS",
  "message": "Inscription validée",
  "username": "user1"
}
```

Requête de Login :

```
{
  "clientId": "uuid-aleatoire",
  "sessionId": "",
  "action": "LOGIN",
  "payload": {
    "username": "user1",
    "password": "mdp"
  }
}
```

Réponse : (renvoie l'id de session)

```
{
  "status": "OK",
  "action": "LOGIN_SUCCESS",
  "message": "Connexion réussie",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Requête demande d'envoie de la liste des QCM :

```
{
  "clientId": "uuid-aleatoire",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "action": "DEMANDE_LIST_QCM",
  "payload":
}
```

Réponse : (envoie de la liste des QCM disponibles) :

```
{
  "status": "OK",
  "action": "QCM_LIST",
  "payload": [
    {
      "qcmId": 1,
      "titre": "Random QCM"
    },
    {
      "qcmId": 2,
      "titre": "Random QCM"
    }
  ]
}
```

Requête demande d'envoi de QCM :

```json
{
  "clientId": "uuid-aleatoire",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "action": "DEMANDE_EXEC_QCM",
  "payload": {
    "qcmId": 1
  }
}
```

Réponse : (envoie question par question) :

```json
{
  "status": "OK",
  "action": "QCM_QUESTION",
  "payload": {
    "qcmId": 1,
    "questionIndex": 1,
    "enonce": "Question n°1 : Quelle est la bonne réponse ?",
    "options": [
      "Réponse 1",
      "Réponse 2",
      "Réponse 3"
    ]
  }
}
```

Requête : Envoie de la réponse à une question du QCM

```json
{
  "clientId": "uuid-aleatoire",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "action": "REPONSE_QCM",
  "payload": {
    "qcmId": 1,
    "questionIndex": 1,
    "optionIndex": 2
  }
}
```

Lorsque l'on arrive à la dernière question, le serveur envoie le Score.

```
{
  "status": "OK",
  "action": "SCORE",
  "payload": {
    "score": 15,
    "totalQuestions": 20
  }
}
```

Requête demande profile :

```
{
  "clientId": "uuid-aleatoire",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "action": "DEMANDE_PROFIL",
  "payload":
}
```

Réponse :

```
{
  "status": "OK",
  "action": "PROFILE",
  "payload": {
    "username": "user1",
    "historique_scores": [
      {
        "qcmId": 1,
        "score": 15
      },
      {
        "qcmId": 3,
        "score": 18
      }
    ]
  }
}
```

Requête suppression du compte :

```
{
  "clientId": "uuid-aleatoire",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "action": "DELETE_USER",
  "payload": {}
}
```

Réponse :

```
{
  "status": "OK",
  "action": "DELETE_USER_SUCCESS",
  "message": "Compte supprimé avec succès. Au revoir !"
}
```

Stockage des QCM :

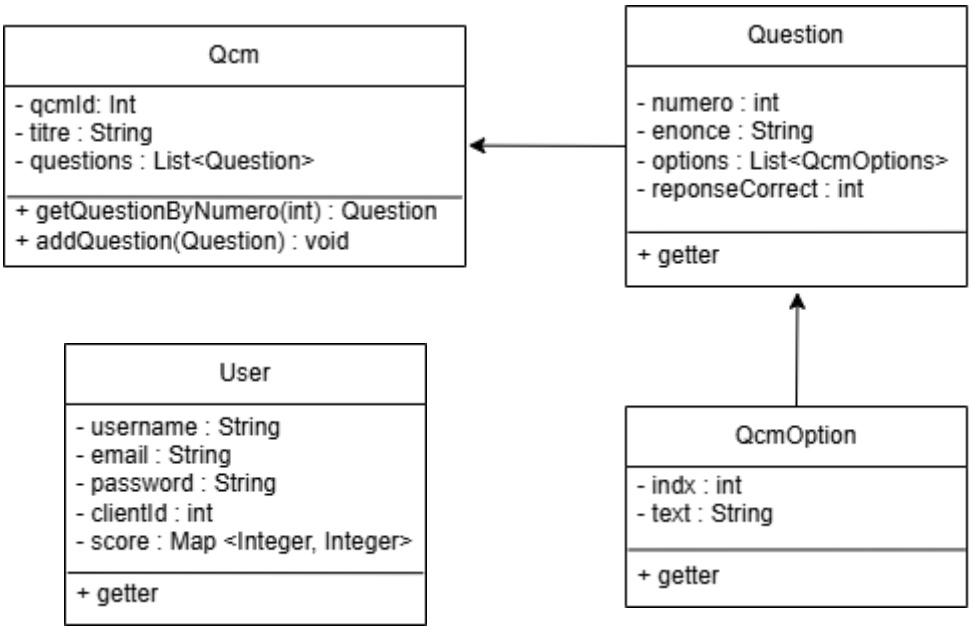
```
[
  {
    "qcmId": 1,
    "titre": "Programmation Java (Niveau 1)",
    "questions": [
      {
        "numero": 1,
        "enonce": "Quelle est l'extension d'un fichier source Java ?",
        "options": [
          {
            "idx": 1,
            "text": ".class"
          },
          {
            "idx": 2,
            "text": ".java"
          },
          {
            "idx": 3,
            "text": ".js"
          }
        ],
        "reponseCorrecte": 2
      },
      ... Autre questions
    ]
  },
  ... Autre QCM
]
```

Stockage des utilisateurs :

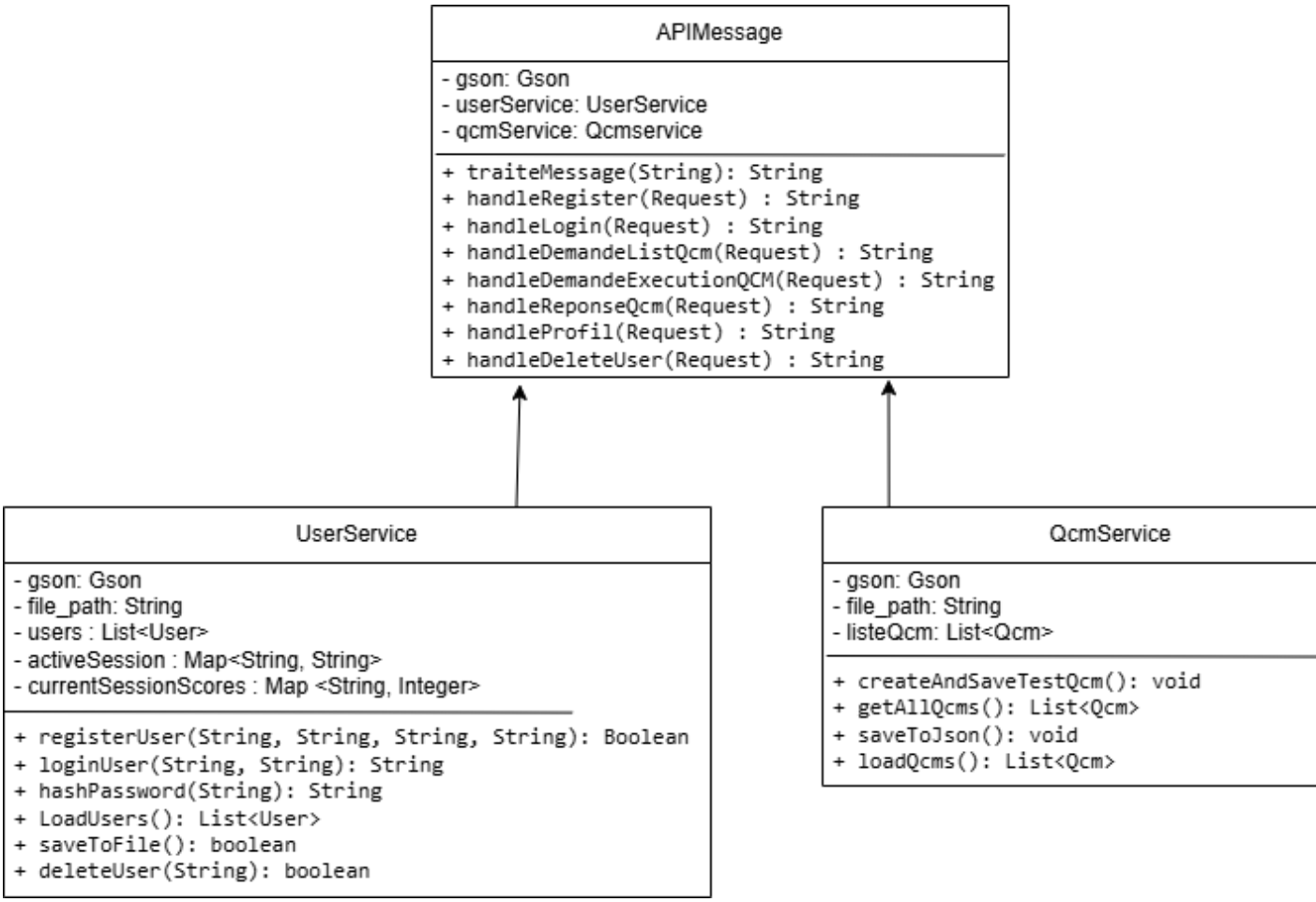
```
[
  {
    "username": "user1",
    "email": "fake@email.fr",
    "userId": "client-id",
    "password": "password",
    "scores": {
      "1": 15,
      "2": 18
    }
  },
  ... autre utilisateur
]
```

C. Diagramme de classes UML :

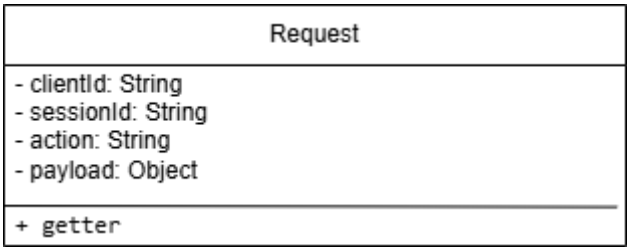
Serveur :



Un Qcm est identifié par son id et possède un titre, il est également composé de plusieurs questions. Les questions sont indexé et possèdent un énoncé, une liste d'option et la réponse à la question. Chaque QcmOption est indexé et possède un texte.

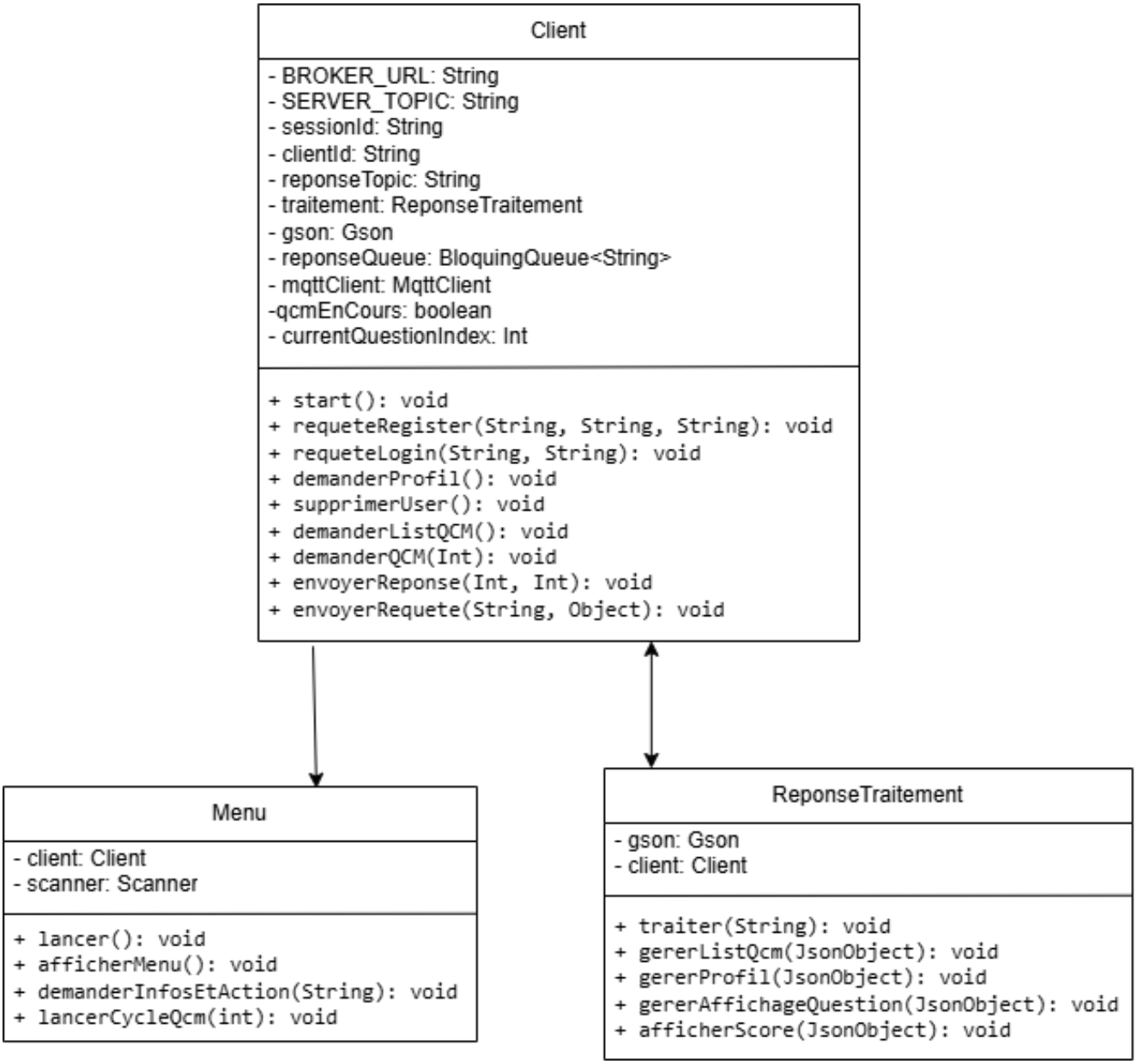


APIMessage utilise UserService pour gérer les utilisateurs, cela va servir pour la correspondance de session, la recherche d'information spécifique à l'utilisateur (profil, suppression compte). Cette classe utilise également QcmService, qui va permettre l'accès aux QCM, cela permet d'envoyer la liste des QCM à l'utilisateur et exécuté un cycle de QCM.



La classe Request a pour but de standardiser le format des échanges. Elle simplifie la construction des messages côté client en encapsulant les données nécessaires, tout en simplifiant l'accès aux attributs de la requête pour le serveur.

Client :



Le client s'abonne au topic clientId/reponse puis lance le menu. Lorsque l'utilisateur choisis une action, le client vas attendre une réponse du serveur (bloquingQueue). Lorsqu'une réponse arrive il appelle la fonction « traiter() » de l'objet ReponseTraitement qui, à l'aide d'un switch case vas agir en fonction de l'action contenu dans le message.

Menu sert a afficher les options disponible à l'utilisateur et à créer les requêtes par le biais du client en fonctions des actions de l'utilisateur.

ReponseTraitement sert à traiter les cas de réponse du serveur, affichage du profil, affichage liste QCM, affichage du Score etc…

3. L’approche TDD pour la mise en place des tests :

Méthode TDD pour l’API :

TDD pour Test-Driven Development consiste à créer les test de fonctions pas encore conçus. Dans mon projet j’ai un dossier test, je vais recréer la même hiérarchie que pour mon vrai code dans lequel se trouve la classe APIMessage. Dedans j’ai créé la classe APIMessageTest qui seras donc le test de cette classe.

Dans celle-ci je vais tester toutes les requêtes possibles envoyer à mon API, register, login, listeQCM, demandeQCM… Le cœur du fonctionnement du serveur passeras par la fonction **traiteMessage(String JsonString)** qui prend en paramètre le message reçu par le serveur. Je vais créer des test qui vont prendre une requête JSON spécifique, et je vais appeler ma fontion traiteMessage, pour vérifier que la requête soit effectivement bien traité par ma fonction.

On vérifie cela en analysant la réponse avec un assertTrue(reponse.contains(« OK »), « L’API traite bien la requête »); ou d’autres assert qui vérifierons le contenu de la réponse créer par exemple : Dans le cas d’une demande de profil, on vérifieras que la réponse contient bien les attribut score et qcmId.

Le but est de développer ma fonction traiteMessage(String jsonString) en allant et de passer les tests.

Par exemple :

Pour commencer on va mettre en place notre base, qui va être de créer un objet userService qui gère les actions lié aux utilisateurs, un objet QcmService qui gère les actions liés aux QCM et un objet APIMessage qui va traiter les requêtes de l’utilisateur. Tous ceci va être créer avant chaque test grâce à l’instruction @BeforeEach.

Exemple pour Register :

Pour le test de Register, je vais créer un JSON avec l’action : « REGISTER », le payload remplis avec un username et un password.

J’appelle la fonction TraiteMessage de l’objet de type API_Message api, je vais tester si la réponse est « OK » et je peux également regarder dans le fichier users.json si l’utilisateur est bien présent.

4. Documentation et implémentation du projet

A. Installation de wget :

```
sudo apt-get install wget
```

B. Installation JDK 25 :

On copie colle l'adresse du lien contenant l'archive du JDK25 pour linux et on utilise :

```
wget lien_de_l_archive
```

Lien d'une archive JDK25 :
https://download.java.net/java/GA/jdk25.0.1/2fbf10d8c78e40bd87641c434705079d/8/GPL/openjdk-25.0.1_linux-x64_bin.tar.gz

Puis il faut la dezip avec :

```
tar -xzf nom_archive.tar.gz
```

Et placer ce que l'on a extrait dans le dossier /opt, avec :

```
sudo mv nom_archive /opt/
```

Et enfin mettre java dans le PATH, ouvrir le fichier ./bashrc (vi ~/.bashrc) et à la fin ajouter ceci :

```
export JAVA_HOME = /opt/nom_archive
export PATH = $JAVA_HOME/bin:$PATH
```

C. Installation de Gradle manuellement :

On récupère le zip gradle bin avec wget :

```
-   wget lien_de_gradle_bin
```

un lien valide : https://services.gradle.org/distributions/gradle-9.2.1-bin.zip

Il faut installer unzip :

```
sudo apt install unzip
```

Puis suivre les instructions trouvé sur le site web de Gradle :

```
$ mkdir /opt/gradle
$ unzip -d /opt/gradle gradle-9.2.1-bin.zip
$ ls /opt/gradle/gradle-9.2.1
```

Et enfin ajouter Gradle au PATH :

```
export GRADLE_HOME =/opt/gradle-9.2.1
export PATH= $GRADLE_HOME/bin:$PATH
```

D. Cloner le projet git dans la machine cliente et serveur :

```
git clone https://gitlab-mi.univ-reims.fr/geer0013/info0502_qcm_joseph_geeraerts.git
cd info0502_qcm_joseph_geeraerts
```

E. Exécuter le projet Serveur :

```
cd QCM_Manger
gradle build
gradle run
```

Le serveur seras alors en attente de traiter des requêtes provenant des clients.

F. Exécuter le projet Client :

```
cd Client1
gradle build
gradle run
```

Le client est lancé, le menu s'affiche et on peut choisir d'envoyer des requêtes.

5. Résultat du jeu d'essai :

Register (r) :

On renseigne le nom d'utilisateur (user2) , l'email (user2@gmail.com) et le mot de passe (mdp)

```
--- BIENVENUE SUR L'APPLICATION QCM ---
[r] S'inscrire (Register)
[l] Se connecter (Login)
-------------------------
1. Afficher mon profil
2. Liste des QCM
3. Demander un QCM
4. Supprimer son compte
0. Quitter

Votre choix :
<===========--Nom d'utilisateur :
<===<===========--> 87% EXECUTING email :
<=<==<===<===<===========--> 87% EXECUTINGMot de passe :
<===========--> 87% EXECUTING [19s]                    Register success
```

Login (l):

On envoie le nom d'utilisateur (user2) et le mot de passe (mdp), on peux voir que le serveur a bien envoyé un sessionId.

```
Votre choix :
<<===========-Nom d'utilisateur :  16s]
<===<<===========--> 87% EXECUTINGMot de passe :
<==<===========--> 87% EXECUTING [1m 27s]        [SYSTEM] Session mise à jour : 8b2c8305-3c97-402a-98ad-1b498b7d1d80
Opération réussie !
```

Afficher profil (1) :

Pour l'instant aucun QCM n'a été effectué

```
--------------------------------
       PROFIL UTILISATEUR
--------------------------------
Nom : user2

--- Historique des Scores ---
Aucun QCM effectué pour le moment.
--------------------------------
```

Demander liste QCM (2) :

```
--- Liste des QCM disponibles ---
- QCM 1 : Random QCM
```

Demander un QCM (3) :

On renseigne l'id du QCM voulu (1), et le serveur envoie alors la première question.

```
<<===========-Entrez l'ID du QCM :s]
<<===========--> 87% EXECUTING [1m 2s]

--- QUESTION N°1 ---
Question n°1 : Quelle est la bonne réponse ?
   [1] Réponse 1
   [2] Réponse 2
   [3] Réponse 3

Entrez le numéro de votre réponse :
```

Répondre au QCM :

Pour répondre au qcm il suffit d'envoyer, 1, 2 ou 3

```
Entrez le numéro de votre réponse :
<<===========--> 87% EXECUTING [1m 59s]
> :run
1
```

Après avoir complété tous le QCM ->

On répond à la dernière question, et le serveur renverras le score final.

```
--- QUESTION N°20 ---
Question n°20 : Quelle est la bonne réponse ?
   [1] Réponse 1
   [2] Réponse 2
   [3] Réponse 3

Entrez le numéro de votre réponse :
<<===========--> 87% EXECUTING [2m 40s]

==================================
       RÉSULTAT DU QCM
       Votre score : 20 / 20
==================================
```

Afficher profil (1) :

Maintenant nous avons le score mis à jour, et la moyenne du score sur le QCM que l'on vient de faire.

```
----------------------------------
       PROFIL UTILISATEUR
----------------------------------
Nom : user2

--- Historique des Scores ---
• Random QCM : 20 pts (Moyenne globale : 15,00)
----------------------------------
```

*

Supprimer utilisateur (4) :

Cela enlève l'utilisateur du fichier users.json, toutes ses données sont supprimés.

```
<===========--supression du compte ...
Compte supprimé avec succès. Au revoir !
Déconnexion en raison de la supression du compte
```